CS408 Individual Project

Progress Report

Digital Logic Simulator

Robert Saarloos (201846521)

November 18, 2020

1 Aims and Objectives

A problem with learning the practical applications of logic is not the quantity of material available, but the lack of a simple, comprehensive guide that takes the student from the basics of digital logic, to simple devices and how they can be connected together to build a basic computer. This project aims to solve this problem by providing a clean and approachable introduction to logic by combining a digital logic simulator and a set of lessons to teach the student how to build a working computer.

The primary goal of this project is to produce a piece of software that acts as a simulator for digital logic circuits. The project's secondary goal is to provide a set of lessons that will teach the user digital logic in a practical and interactive way. The final result of this project will be a tool that acts as an all-in-one practical guide on how to produce a very simple, Turing-complete computer, based on the von Neumann architecture. This tool will be targeted at both students and casual learners looking for an interactive introduction to computational logic. The program should be able to teach someone, who knows nothing or little about digital logic, step-by-step how to build increasingly complex digital devices that will be combined to produce a functioning computer. This will be purely in terms of logic, without requiring the user to learn about electronic engineering during the process.

The logic simulator will provide a simple, grid-based environment in which users can place wires, logic gates, and circuit packages. The circuit packages and logic gates will be similar to semiconductor packages, using pins to represent the inputs and outputs of the packages. Wires will be used to connect these pins to carry signals between them. Wires will be organised by coloured layers, meaning that wires of differing color cannot directly pass signals between each other. This allows for the creation of complex, layered wiring, but with clear visual distinction between different layers.

Each lesson will provide the user with two circuit packages, one will provide a set of pre-defined inputs, the other will take a set of outputs. The purpose of these packages will be to test that the user's circuit works as expected. If the received outputs are correct, the user will be able to progress to the next lesson. Each lesson will also provide a briefing note, outlining the task and provide the user with the information required to complete the task.

The program will also provide a "sandbox" mode in which there is no set objective, and the user is free to experiment with the simulator and make any circuit they like. All features that may be restricted in the lessons will be available to the user, and any elements that are unique to the lessons will be removed.

2 Background Information

2.1 Digital Logic

Digital logic is a subset of logic that specifically deals with binary values (true/false, on/off, etc.). This type of logic provides us with a few elementary operators to act on binary values, much like arithmetic provides us with operators for addition and subtraction. These digital logic operators are AND, OR, XOR and NOT. Each of these operators takes a set of binary values as inputs and transforms them to produce a single value as an output. Combining these operators allows us to build more complex transformations. We refer to this as a logic "circuit". We can represent the behaviour of each of these operators as a truth table. A truth table shows every input permutation for each operator and the outputs that it would produce. These are the truth tables for our basic operators:

AND		
A	B	Output
T	T	T
T	F	F
F	T	F
F	F	F

OR		
A	B	Output
T	T	T
T	F	T
F	T	T
F	F	F

XOR		
A	B	Output
T	T	F
T	F	T
F	T	T
F	F	F

NOT	
Input	Output
T	F
F	T

Digital logic, in terms of computers, concerns the use of the binary number system. In the context of the project, digital logic refers to the use of combinatorial and sequential logic to perform logical transformations on some input to produce some output. Most practical digital devices are a combination of combinatorial and sequential logic.

Combinatorial logic encompasses some form of digital logic circuit, or boolean equation, that directly transforms some input to an output, where the internal state of the circuit has no effect on the output. A defining characteristic of combinatorial logic circuits is that they can be represented as a truth table.

Sequential logic, in contrast, refers to a digital logic circuit where the output can be influenced by the internal state of circuit. This means that the circuit is dependent not only on the current input, but every input previously received. The "internal state" of the digital logic circuit primarily refers to memory. Circuits with memory hold their state across different input sets and, depending on circuit, the same input may not produce the same output every time. Therefore sequential logic circuits cannot be represented as a truth table.

An important property of digital logic circuits is their ability to be synchronised with other circuits by a clock signal. A clock is simply a circuit that generates an oscillating signal with a constant frequency. In electronic circuits, this is most often achieved through the use of the piezoelectic properties of quartz to generate a consistent electrical pulse.

Circuits that are able to be synchronised are designed to take a clock signal as an additional input, and will only update their state when either the clock signal transitions from low to high, or high to low. Most synchronous circuits are designed to update on only one of these transitions. If a circuit updates its state on the clock signal changing to a high signal or changing to low signal, it is said to be a rising-edge or falling-edge circuit, respectively. The ability to control which edge of a clock signal a circuit updates its state on is important for building a digital computer.

2.2 Turing Machine

Since this project will culminate in the user building a simple computer, it is worth exploring what a computer is in more concise terms. The most well known theoretical model of computation is the Turing machine, which describes an infinitely long tape of memory, subdivided into cells, each holding a "symbol". The machine has a read/write head that can read a symbol from a cell or write a symbol to a cell. Each symbol represents an operation defined by a finite table of user-specified instructions. The machine would read the symbol at its current position, execute that instruction, write a symbol to the tape based on that instruction, and then advance to the next cell, forming the basis of a fetch, decode, execute cycle.

The purpose of the Turing machine is to describe a mechanism for computation which, given any algorithm, a Turing machine can be created to simulate the logic of that algorithm. A machine capable of emulating any turing machine is said to be "Turing-complete". A Turing machine as it is described here is not practical for computation, but it provides a minimal model of the mechanism for what our machine should be able to achieve.

2.3 Von Neumann Architecture

The Von Neumann architecture is a model for a computer that reduces the overall structure of the machine to a few simple components: A central processing unit, a memory unit, an input device and an output device. The central processing unit is comprised of a control unit, an arithmetic logic unit, and registers. The control unit consists of an instruction register and a program counter. The memory unit serves to store data and instructions.

An interesting thing to note about the way memory is handled in the Von Neumann architecture is that instructions and data are stored on the same memory device. This means that the machine's instructions and data are not strictly distinct, and so instructions can be treated as data. This grants opportunities for software development using self-modifying code, a useful tool for when computer memory was extremely limited. This is also what gives compilers of high level languages the ability to operate, as they treat the instructions as data, which may then be executed as a set of instructions when compiled.

Our machine will be a reduced form of the Von Neumann architecture, in that the input and output devices will, for the most part, be non-existent, aside from manually controlling the state of the machine's inputs. However, this is still enough to build a machine capable of emulating a Turing machine, and so it is Turing-complete.

2.4 Digital Computers

Early computers were a mix of mechanical, electro-mechanical and electronic machines. Most of these machines were designed with a singular application in mind, and were not programmable, falling into the category of "finite-state automata". Before a digital, binary, general-purpose computer was invented, there were a variety of efforts to build a programmable general-purpose computing machine. Some of these machines worked on a decimal model, rather than binary, however this proved impractical for implementation using an electronic medium, as it is much more difficult to accurately stablise ten voltage levels as opposed to only two, and therefore less reliable.

The first true programmable general-purpose machine is said to be the ENIAC (Electronic Numerical Integrator and Computer), however "programmable" is perhaps too strong a word, as the machine's instructions were determined by plugboards, and programming these instructions would take days of rewiring the whole machine. Despite this, the machine was entirely electronic, eliminating the need for punch cards and was the fastest and most powerful computer built at the time. John von Neumann became involved with the ENIAC and, through this, designed a stored-program computer architecture that would eliminate the need for punch cards and plugboards.

By this point, the switch to purely electronic machines was well underway. More advanced computers with greater capabilities continued to be developed as the architectures of computing systems started to converge to a more predictable standard. The mediums which implemented logic gates started evolving too, vacuum tube triodes gave way to silicon semiconductor transistors, these silicon semiconductors were able to be made smaller and smaller until they could be built on tiny shards of silicon. Entire processors could then be

set into a package smaller than a hand, affordable and appropriate for commercial use. A far cry from the ENIAC and its similarly massive successors.

2.5 Related Projects

There are many pieces of software that tackle this problem domain, at least in terms of the simulator. Some do this better than others, and some have solved the common issues that are unique to these types of simulators.

The first is Boolr, a Dutch high school project implemented in Javascript. The notable attraction of this program is its clean interface and visual style. Despite its sharp, modern look it had several problems. The main issue is with small cyclic circuits, such as flip-flops, which could oscillate under certain circumstances, toggling the flip-flop very rapidly and not being able to settle its state. Another issue with Boolr is the storage of circuits. The circuits are stored in an XML format. In a test build of a small 8-bit machine, the project file quickly bloated to several gigabytes in size. It is not known if this is a fault with the format itself, or simply poor use in the program's implementation. After a point, the file became so large that the program could no longer load it, and doing so would cause the program to lock. The main takeaway from Boolr would be its simplicity and visual style. It would be best for there to be as little unnecessary visual clutter in the application as possible. The user presumably wants to learn about digital logic, so asking them to learn a user interface reminiscent of a airplane dashboard seems counter-intuitive.

The second piece of software we will look at is Digital, a digital logic simulator with a massive library of features, including a set of analysis tools, such as representing the circuit as a truth table or a Karnaugh map. It is a spiritual successor to Logisim and solved many of the issues associated with it, such as circuit settling to put the circuit in a stable state before simulating it. Digital touts impressive performance capabilities, estimating that a simple processor can be simulated with a 120kHz clock. The developer's description of some core software features will inform some of the design decisions made in this project, specifically the idea of an event based simulator. The developer does not go into detail about what is meant by "event based", but it inspires some attractive ideas about how to approach the problem of ensuring that the simulator's logic is consistent and correct.

Digital and Boolr share some common useful features, the main one being the ability for the user to abstract circuits into black-box circuit packages. This is an invaluable tool in building large digital circuits, especially when building something with as many interdependent components as a computer.

3 Requirements Specification

- Users should have a choice to start the simulator in their preferred mode.
  - Lesson mode: Prescribed course of lessons.
  - Sandbox mode: Free use of all simulator features with no prescribed objectives.
- Large board on which the user can build circuits
  - Board should be large enough so as not to restrict the user.
  - Users should be able to zoom in on parts of the board.
  - Control scheme should be intuitive.
  - Users should be able to move to different parts of the board.
- Users should be able to build circuits
  - Logic gates the user can place on the board.
  - Wires that are used to connect logic gates together.
- Logic gates as the elementary operators for building digital circuits.
  - NOT
  - OR
  - AND
  - XOR
- The behaviour of circuits should be simulated, so the user can run and test their creations.
  - The state of the circuit should be predictable and consistent.
  - Circuit state should be persistent so that memory circuits reliably hold stored data.
  - The circuit simulation must be fast enough to process large circuits quickly enough so as not to frustrate users.
  - The circuit simulation must be stable and robust.
  - The simulation should be capable of detecting problematic circuit configurations.
  - Users must receive a warning when an erroneous circuit configuration is detected.

- Users should be able to change and see the state of their circuit via input and output nodes.
  - Input nodes will act as a user-operated input to their circuit.
  - Output nodes will act as an output for the user's circuit.
  - Both nodes should visually indicate their current state.
- The state of wires should also be indicated (highlighted when carrying an "on" signal).
- Users should be able to save their current session.
  - Both circuit structure and state must be saved so that memory circuit contents are preserved.
- Users should be able to load a previously saved session.
  - Loaded circuits should be identical in structure and state to the saved circuit.
- The program should be capable of dealing with erroneous save files.
- Users should be able to import saved circuits for use in a separate session.
- Saved circuits should be able to be named.
- A set of lessons capable of teaching users how to build a simple computer.
  - Introduction, logic gates and lesson structure.
  - Adders
  - Decoders
  - Flip-flops
  - Memory
  - Counters and clocks
  - Arithmetic logic unit
  - General purpose registers
  - Flag registers and program counter
  - Turing-complete machine

4 Project Plan

4.1 Prototype

At the time of writing, the project is at a point where the following features have been fully implemented:

- Large board on which users can build circuits
  - Users can navigate the board using the keyboard.
  - Users can use the mouse to place wires on the board.
  - The world is large enough so as not to restrict users, even when building extremely large circuits.
- Wires
  - Connects circuit package pins and allows transmission of signals between them.
  - Organised in layers (denoted by color).
  - Wires on seperate layers do not connect to each other.
- Toolbar providing the user with a set of tools that they will frequently need.
  - NOT gate
  - OR gate
  - AND gate
  - XOR gate
  - Three colors of wire (red, green and blue)

4.2 Development timeline

The intended course of development will follow this timeline:

- November 2020
  - Circuit packages
- December 2020
  - Logic gates
  - Custom circuit packages

  - Circuit simulation
  - Wire highlighting
  - World serialisation
- January 2021
  - Utility buttons
    * New circuit
    * Save circuit
    * Load circuit
    * New circuit package
  - Lessons
    * Introduction, logic gates and lesson structure
    * Adders
    * Decoders
    * Flip-flops
    * Memory
- February 2021
  - Lessons
    * Counters and clocks
    * Arithmetic and logic unit
    * General purpose registers
    * Flag registers and program counter
    * Turing-complete machine
  - Final testing pass
- March 2021
  - Evaluation
  - Final project report

5 Methodology

In a graphical application such as this, where back-end logic must be performant, but also fast to transform into renderable data, there is a demand for a primary focus on the performance of the application. For this reason, the design for this project will focus on how the data itself should be transformed and less on how it is to be represented structurally. With that, the project will be designed with some data-oriented design principles in mind, along with some object-oriented design patterns where appropriate. The reason for this decision is for speed, especially in parts of the program where a lot data will need to be accessed in series, such as tile states, that could be naturally blocked together in a contiguous memory block. Keeping these packaged together and focusing on how the data relates to each other is good way of producing logical groupings of data that benefit the flow of data in the application, as opposed to trying to relate it as closely as possible to the problem domain. Another reason is that it also allows more flexibility in the case of high-level design changes that may otherwise require a lot of change to the implementation. Additionally, circumstances in which certain stages of the pipeline can be processed concurrently will be more clear, and easier to implement.

5.1 Implementation

Implementation of the project will be done in C. The decision for using this language is multi-faceted. For one, it meshes well with the chosen design paradigm. The game engine and tools that will be used in implementing the application are also written in C. There was some consideration put into using C++, however this would make little difference in terms of how the program is implemented, but certain type-checking standards of the language would become intrusive. Implementation will closely follow the design, while employing some techniques and practices that are common in ensuring good performance, robustness and maintainability: branchless programming, pure functions and flattening, to name a few.

5.2 Testing

Testing, on a more granular scale, will involve unit tests as the software is developed. The structure of the software and the design principles employed will make this process very simple, so long as the associated structures exist, even in a rudimentary sense. As modules are built and connected to each other, the communication of data between them will be

evaluated in terms of correctness and performance. Regular checks will be made for memory leaks and performance issues as the software is developed. At a higher level, later in development, the correctness of the logic simulator will be able to be assessed through the lesson system itself. Using the two input/output circuits to supply and verify test data and output to prove that the circuit simulator operates as intended and is fit for purpose.

5.3 Evaluation

In evaluating the software, the primary question that needs answering is "does this software teach users how a computer works." It will take time to get an honest answer to this question, and will require participation from people who are not particularly well versed in computational logic. It will not be easy for participants to provide this data, as it will require them to complete the lesson series in its entirety, so some level of interest in the subject will be necessary. It appears possible that, if well thought out, the lessons themselves could act as an evaluation of the user's understanding, and completion of the lessons would be proof of the software's effectiveness in teaching the user how a computer works. Some feedback through a light questionnaire could prove useful in verifying that the software achieves its intended goal, and potentially highlight unforseen issues. Subjecting the software to several different users will also help establish its reliability and robustness.

5.4 Tools

Each stage of development will make use of a set of tools. The most significant are those used in implementation: An in-house game engine, using OpenGL, GLEW, SDL and CGLM. Additional tools are used for evaluating the software's memory management and performance: GDB for debugging; Valgrind for tracking memory leaks; and Heaptrack for additional evaluation of memory metrics.